

Strings and String Operations

In Module 1, we learned how to create variables, take input from the user, perform calculations, and display information using `print()`.

In this module, we will learn more about **strings**.

A string is text stored inside a Python program. Strings are extremely important because almost every useful program works with text in some form.

In this module, we will learn:

- What strings are
- Single and double quotation marks
- Storing strings in variables
- Combining strings
- String repetition
- Finding the length of a string
- Accessing individual characters
- String indexing
- Negative indexing
- String slicing
- Useful string methods
- Changing uppercase and lowercase
- Removing unnecessary spaces
- Replacing text
- Checking whether text is present
- Working with user input
- Building simple text-based programs

What is a String?

A string is a sequence of characters.

Characters can include:

- Letters
- Numbers
- Spaces
- Symbols
- Special characters

Examples:

```
1 "Hello"  
2 "Python"  
3 "Guwahati"  
4 "12345"  
5 "Hello World"  
6 "Python 3.14"
```

Even though 12345 contains numbers, it becomes a string when it is written inside quotation marks.

Compare:

```
1 number = 12345  
2 text = "12345"
```

The first value is an integer.

The second value is a string.

This difference becomes important when performing calculations.

Quotation Marks

Python allows us to create strings using either single quotation marks or double quotation marks.

Both are valid:

```
1 name = "Rahul"  
2 city = 'Guwahati'
```

There is no difference in the type of string created.

Both are strings.

```
1 print(type("Hello"))
2 print(type('Hello'))
```

Both produce:

```
1 <class 'str'>
2 <class 'str'>
```

Which Should You Use?

Choose one style and use it consistently.

For this course, we will generally use double quotation marks:

```
1 name = "Rahul"
2 city = "Guwahati"
3 course = "Python"
```

Storing Strings in Variables

Strings can be stored in variables just like numbers.

```
1 name = "Rahul"
2 city = "Guwahati"
3 course = "Python"
4
5 print(name)
6 print(city)
7 print(course)
```

Output:

```
1 Rahul
2 Guwahati
3 Python
```

A variable can also be changed:

```
1 city = "Guwahati"
2
```

```
3 print(city)
4
5 city = "Nagaon"
6
7 print(city)
```

Output:

```
1 Guwahati
2 Nagaon
```

Combining Strings

Strings can be joined using the + operator.

This is called **string concatenation**.

Example:

```
1 first_name = "Rahul"
2 last_name = "Sharma"
3
4 full_name = first_name + last_name
5
6 print(full_name)
```

Output:

```
1 RahulSharma
```

There is no space because Python simply joined the two strings.

To insert a space:

```
1 first_name = "Rahul"
2 last_name = "Sharma"
3
4 full_name = first_name + " " + last_name
5
6 print(full_name)
```

Output:

```
1 Rahul Sharma
```

The string " " contains one space.

Strings and Numbers

We cannot directly join a string and an integer using `+`.

For example, this produces an error:

```
1 age = 20
2
3 print("Age: " + age)
```

The problem is that `"Age: "` is a string while `age` is an integer.

We can convert the number to a string using `str()`:

```
1 age = 20
2
3 print("Age: " + str(age))
```

Output:

```
1 Age: 20
```

An easier method is often to use commas:

```
1 age = 20
2
3 print("Age:", age)
```

Output:

```
1 Age: 20
```

Repeating Strings

The multiplication operator can be used with strings.

For example:

```
1 print("Hello" * 3)
```

Output:

```
1 HelloHelloHello
```

If we want spaces between the repeated words:

```
1 print("Hello " * 3)
```

This produces:

```
1 Hello Hello Hello
```

The final output contains a trailing space because the original string contained a space after Hello.

Another example:

```
1 print("*" * 10)
```

Output:

```
1 *****
```

This can be useful for creating simple lines or separators.

Finding the Length of a String

Python provides the `len()` function to find the number of characters in a string.

Example:

```
1 name = "Rahul"  
2  
3 print(len(name))
```

Output:

```
1 5
```

The five characters are:

R, a, h, u, l

Spaces are also counted.

For example:

```
1 text = "Hello World"  
2  
3 print(len(text))
```

Output:

```
1 11
```

There are 10 letters plus one space.

String Indexing

A string is a sequence of characters.

Python assigns a position, called an **index**, to each character.

Important: Python starts counting from **0**, not 1.

Consider:

```
1 word = "PYTHON"
```

The positions are:

Index	0	1	2	3	4	5
Character	P	Y	T	H	O	N

Therefore:

```
1 word = "PYTHON"
2
3 print(word[0])
4 print(word[1])
5 print(word[2])
```

Output:

```
1 P
2 Y
3 T
```

Why Does Python Start at 0?

Many programming languages use zero-based indexing.

Therefore, the first character is at index 0, the second character is at index 1, and so on.

This is an important concept that will be used throughout Python programming.

Negative Indexing

Python also allows us to access characters from the end of a string using negative indexes.

For:

```
1 word = "PYTHON"
```

the positions are:

Index	-6	-5	-4	-3	-2	-1
Character	P	Y	T	H	O	N

Therefore:

```
1 word = "PYTHON"
2
3 print(word[-1])
4 print(word[-2])
```

Output:

```
1 N
2 O
```

The index `-1` always refers to the last character.

String Slicing

Slicing allows us to extract a part of a string.

The basic syntax is:

```
1 string[start:end]
```

The starting position is included, but the ending position is not included.

Consider:

```
1 word = "PYTHON"
2
3 print(word[0:3])
```

Output:

```
1 PYT
```

Indexes 0, 1, and 2 are included.

Index 3 is not included.

More Slicing Examples

```
1 word = "PYTHON"
2
3 print(word[0:2])
4 print(word[2:5])
5 print(word[1:4])
```

Output:

```
1 PY
2 THO
3 YTH
```

Slicing From the Beginning

We can leave the starting index empty.

```
1 word = "PYTHON"
2
3 print(word[:3])
```

Output:

```
1 PYT
```

This means:

Start from the beginning and stop before index 3.

Slicing Until the End

We can also leave the ending index empty.

```
1 word = "PYTHON"
2
3 print(word[2:])
```

Output:

```
1 THON
```

String Methods

Python provides many built-in methods for working with strings.

A method is an operation that can be performed on an object.

For example:

```
1 name = "rahul"
2
3 print(name.upper())
```

Output:

```
1 RAHUL
```

Notice the dot:

```
1 name.upper()
```

The method `upper()` operates on the string stored in `name`.

Changing Letter Case

`upper()`

The `upper()` method converts letters to uppercase.

```
1 text = "python programming"
2
3 print(text.upper())
```

Output:

```
1 PYTHON PROGRAMMING
```

`lower()`

The `lower()` method converts letters to lowercase.

```
1 text = "PYTHON PROGRAMMING"
2
3 print(text.lower())
```

Output:

```
1 python programming
```

title()

The **title()** method capitalizes the first letter of each word.

```
1 text = "python programming course"
2
3 print(text.title())
```

Output:

```
1 Python Programming Course
```

Removing Extra Spaces

Sometimes information entered by a user contains unnecessary spaces.

For example:

```
1 name = "   Rahul   "
2
3 print(name)
```

The **strip()** method removes spaces from the beginning and end.

```
1 name = "   Rahul   "
2
3 clean_name = name.strip()
4
5 print(clean_name)
```

Output:

```
1 Rahul
```

The **strip()** method does not remove spaces between words.

For example:

```
1 text = "Hello World"
2
3 print(text.strip())
```

The space between **Hello** and **World** remains.

Replacing Text

The `replace()` method can replace one piece of text with another.

Example:

```
1 text = "I like Java"
2
3 new_text = text.replace("Java", "Python")
4
5 print(new_text)
```

Output:

```
1 I like Python
```

The original variable does not automatically change.

For example:

```
1 text = "I like Java"
2
3 text.replace("Java", "Python")
4
5 print(text)
```

The output is still:

```
1 I like Java
```

To keep the changed version, store it in a variable:

```
1 text = "I like Java"
2
3 text = text.replace("Java", "Python")
4
5 print(text)
```

Finding Text

The `find()` method searches for text inside a string.

Example:

```
1 text = "Python Programming"
2
```

```
3 print(text.find("Python"))
```

Output:

```
1 0
```

This means that Python starts at index 0.

Another example:

```
1 text = "Python Programming"
2
3 print(text.find("Programming"))
```

Output:

```
1 7
```

If the text cannot be found, `find()` returns:

```
1 -1
```

For example:

```
1 text = "Python"
2
3 print(text.find("Java"))
```

Output:

```
1 -1
```

Checking Whether Text Exists

The `in` operator can be used to check whether some text exists inside another string.

Example:

```
1 text = "Python Programming"
2
3 print("Python" in text)
```

Output:

```
1 True
```

Another example:

```
1 text = "Python Programming"
2
3 print("Java" in text)
```

Output:

```
1 False
```

This is useful when we need to check information provided by a user.

Counting Characters

The `count()` method tells us how many times a particular character or piece of text appears.

Example:

```
1 text = "banana"
2
3 print(text.count("a"))
```

Output:

```
1 3
```

Another example:

```
1 text = "Python Python Python"
2
3 print(text.count("Python"))
```

Output:

```
1 3
```

Working With User Input

String methods become particularly useful when working with input.

For example:

```
1 name = input("Enter your name: ")
2
3 name = name.strip()
4
5 print("Hello", name)
```

If the user accidentally enters spaces before or after their name, `strip()` removes them. We can also convert the input to uppercase:

```
1 name = input("Enter your name: ")
2
3 name = name.strip()
4 name = name.upper()
5
6 print("Hello", name)
```

If the user enters:

```
1 rahul
```

the output is:

```
1 Hello RAHUL
```

Using Multiple String Methods

We can use more than one method on the same string.

For example:

```
1 name = "   rahul sharma   "
2
3 name = name.strip()
4 name = name.title()
5
6 print(name)
```

Output:

```
1 Rahul Sharma
```

We can also write:

```
1 name = "   rahul sharma   "
2
3 name = name.strip().title()
4
5 print(name)
```

This performs both operations.

Formatted Strings

Python provides a convenient way to place variables inside text using **f-strings**.

Example:

```
1 name = "Rahul"
2 age = 20
3
4 print(f"My name is {name}.")
5 print(f"I am {age} years old.")
```

Output:

```
1 My name is Rahul.
2 I am 20 years old.
```

The letter **f** before the quotation mark tells Python that the string is formatted.

Variables are placed inside curly brackets.

```
1 print(f"My name is {name} and I am {age} years old.")
```

This is generally easier to read than repeatedly using the **+** operator.

Practice Exercises

Attempt each exercise yourself before looking at the answers.

Exercise 1: Basic String

Create a variable called **message** and store:

```
1 Welcome to Python Programming
```

Display the message.

Exercise 2: String Length

Create the following variable:

```
1 word = "Programming"
```

Display the number of characters in the word.

Exercise 3: First and Last Character

Create:

```
1 word = "Python"
```

Display:

- The first character
- The last character

Exercise 4: Slicing

Create:

```
1 word = "PROGRAMMING"
```

Use slicing to display:

- The first four characters
- The last four characters
- Characters from index 2 to index 6

Exercise 5: Uppercase

Ask the user to enter a sentence.

Display the sentence completely in uppercase.

Exercise 6: Lowercase

Ask the user to enter a sentence.

Display the sentence completely in lowercase.

Exercise 7: Name Cleaning

Ask the user to enter their name.

Remove unnecessary spaces from the beginning and end and display the cleaned name.

Exercise 8: Title Case

Ask the user to enter their full name in lowercase.

Convert it into title case.

For example:

```
1 Enter your name: rahul sharma
2 Rahul Sharma
```

Exercise 9: Replace

Create:

```
1 sentence = "I am learning Java"
```

Replace Java with Python.

Exercise 10: Count

Create:

```
1 word = "programming"
```

Count how many times the letter `m` occurs.

Exercise 11: Search

Ask the user to enter a sentence.

Ask the user to enter a word to search for.

Use the `in` operator to determine whether the word exists in the sentence.

Exercise 12: Formatted Output

Ask the user for:

- Name
- Age
- City
- Course

Display the information using an f-string.

For example:

```
1 Name: Rahul
2 Age: 20
3 City: Guwahati
4 Course: Python
```

Answers to Practice Exercises

Answer 1

```
1 message = "Welcome to Python Programming"
2
3 print(message)
```

Answer 2

```
1 word = "Programming"
2
3 print(len(word))
```

Output:

```
1 11
```

Answer 3

```
1 word = "Python"
2
3 print(word[0])
4 print(word[-1])
```

Output:

```
1 P
2 n
```

Answer 4

```
1 word = "PROGRAMMING"
2
3 print(word[:4])
4 print(word[-4:])
5 print(word[2:7])
```

Output:

```
1 PROG
2 MING
3 OGRAM
```

Answer 5

```
1 sentence = input("Enter a sentence: ")
2
3 print(sentence.upper())
```

Answer 6

```
1 sentence = input("Enter a sentence: ")
2
3 print(sentence.lower())
```

Answer 7

```
1 name = input("Enter your name: ")
2
3 name = name.strip()
4
5 print(name)
```

Answer 8

```
1 name = input("Enter your name: ")
2
3 name = name.strip().title()
4
5 print(name)
```

Answer 9

```
1 sentence = "I am learning Java"
2
3 sentence = sentence.replace("Java", "Python")
4
5 print(sentence)
```

Output:

```
1 I am learning Python
```

Answer 10

```
1 word = "programming"
2
3 print(word.count("m"))
```

Output:

```
1 2
```

Answer 11

```
1 sentence = input("Enter a sentence: ")
2 word = input("Enter a word to search: ")
3
4 print(word in sentence)
```

Answer 12

```
1 name = input("Enter your name: ")
2 age = int(input("Enter your age: "))
3 city = input("Enter your city: ")
4 course = input("Enter your course: ")
5
6 print(f"Name: {name}")
7 print(f"Age: {age}")
8 print(f"City: {city}")
9 print(f"Course: {course}")
```

Mini Project: Student Profile

Create a program that asks the user for:

1. Full name
2. City
3. Course
4. Favourite programming language

The program should:

- Remove unnecessary spaces from the name.
- Convert the name into title case.
- Convert the course name into uppercase.
- Display all information using f-strings.

Example:

```
1 Enter your name:    rahul sharma
2 Enter your city: Guwahati
3 Enter your course: python
4 Enter your favourite programming language: python
5
6 Student Profile
```

```
7  Name: Rahul Sharma
8  City: Guwahati
9  Course: PYTHON
10 Favourite Language: Python
```

Try to write the program without looking at the previous examples.

Answer: Student Profile

One possible solution is:

```
1  name = input("Enter your name: ")
2  city = input("Enter your city: ")
3  course = input("Enter your course: ")
4  language = input("Enter your favourite programming language: ")
5
6  name = name.strip().title()
7  city = city.strip().title()
8  course = course.strip().upper()
9  language = language.strip().title()
10
11 print()
12 print("Student Profile")
13 print(f"Name: {name}")
14 print(f"City: {city}")
15 print(f"Course: {course}")
16 print(f"Favourite Language: {language}")
```

Final Practice Task

Create a simple program that asks the user to enter a sentence.

The program should display:

- The original sentence
- The sentence in uppercase
- The sentence in lowercase
- The number of characters

- The number of times the letter **a** appears

It should also ask the user for a word and tell whether that word exists in the sentence.

Answer

```
1 sentence = input("Enter a sentence: ")
2 word = input("Enter a word to search: ")
3
4 print()
5 print("Original:", sentence)
6 print("Uppercase:", sentence.upper())
7 print("Lowercase:", sentence.lower())
8 print("Number of characters:", len(sentence))
9 print("Number of 'a' characters:", sentence.lower().count("a"))
10 print("Word found:", word in sentence)
```

Module Review

Before moving to the next module, make sure you understand:

- What a string is.
- How to create a string.
- The difference between a string and a number.
- How to concatenate strings.
- How to repeat a string.
- How `len()` works.
- What string indexing means.
- Why Python starts indexing at 0.
- How negative indexing works.
- How string slicing works.
- How `upper()` and `lower()` work.

- How `title()` works.
- How `strip()` works.
- How `replace()` works.
- How `find()` works.
- How the `in` operator works.
- How `count()` works.
- How to use string methods with user input.
- How to use f-strings.

End of Python Module 2